

Report 1

Andrei Petrea

Faculty of Automatic Control and Computers

National University of Science and Technology POLITEHNICA Bucharest

June 13, 2026

Abstract

This report examines container image minimization in the context of modern day use-cases. It provides an overview of the state-of-the-art tools and techniques for minimizing container images, as well as a detailed description of my work in developing a new tool for this purpose. The report concludes with a comparison of my tool with existing solutions and a discussion of potential future work in this area.

1 Introduction

It has been almost a decade since the famous *Attention Is All You Need* [1] paper was published, which began the era of artificial intelligence (AI) development, culminating in the creation of large language models (LLMs) such as ChatGPT.

In the past few years, LLMs have gradually transitioned from simple chatbots into highly sophisticated tools that can perform a wide range of tasks. This transition is characterized by a migration away from simple prompt-and-response dynamics toward continuous, goal-oriented autonomy. Modern agentic frameworks leverage the underlying model not just for linguistic fluency, but as a dynamic engine for systemic interaction—capable of maintaining context over extended horizons, utilizing specialized tools, and executing complex, multi-stage workflows within broader software architectures.

As such, it is natural to explore how LLMs can be utilized to build minimal container images. In this context, I created an agentic workflow where LLMs are leveraged to fuzz the application and in order to find as many execution paths in order to more accurately determine the minimal set of dependencies required to run and interact with the application.

2 State of the Art

There are two main approaches to minimizing container images:

- **Use Smaller-sized Libraries:** This approach involves replacing larger libraries with smaller alternatives that provide the same functionality.
- **Removing Unnecessary Components:** This approach focuses on stripping away non-essential components from the container image.

Let us explore some of the ways these approaches are implemented in practice.

Slim Debian

Slim Debian is a stripped-down version of the standard Debian image. It removes unnecessary components and libraries, resulting in a smaller image size compared to the full Debian image (29 MB vs 47 MB). However, the reduction is negligible compared to the original image, and it still includes a lot of components that may not be necessary for all applications, such as package managers and shells.

Alpine Linux

One of the most popular base images for minimizing container size is *Alpine Linux*. Instead of the standard *glibc* library, which is 2.2MB in size, it uses the *musl* library (approximately 650KB). As such, any library that depends on *glibc* can be replaced with a version that relies on *musl*, resulting in a significant reduction in size. Most popular frameworks and runtime environments, such as Python, Node.js, and Java, have Alpine-compatible versions that utilize *musl*. In order to further reduce the size, Alpine Linux employs *Busy-Box*, a software suite that provides several Unix utilities in a single executable file. The main disadvantage of using Alpine Linux is compatibility. In order to use it, developers must ensure that their applications and dependencies are compatible with *musl*, which may prove impossible for complex applications.

Minimal Images

Minimal images, frequently referred to as *distroless* images, are streamlined container environments that exclude non-essential system components such as package managers, shells, and standard operating system distributions, retaining only the application and its runtime dependencies [?]. As they only include the bare essentials required to run the application, they have the most reduction in size compared to the other approaches, however they are the most difficult to create and maintain, as they require a deep understanding of the application's dependencies and careful management of updates and security patches.

Building Minimal Images

Building minimal images is intrinsically linked to finding the dependencies of the application and extracting them into a new container image. This process can be achieved through various techniques, such as:

- **Static Analysis:** analyzing the application's code and configuration files to identify its dependencies

- **Dynamic Analysis:** executing the application and monitoring its behavior to identify the dependencies it uses at runtime
- **Brute Force:** iteratively removing components from the container image and testing if the application still functions correctly, until a minimal set of dependencies is identified

These techniques can be done manually, but they are time-consuming and error-prone, especially for complex applications with many dependencies. As such, there is a need for automated tools that can streamline this process and generate minimal container images efficiently and accurately.

Let us examine some of these tools.

Bazel

Bazel is a build automation tool created by Google that supports building and testing software applications [?]. It includes support for building Docker images through its *rules_oci* extension and it made the first distroless images. It uses a file called *BUILD* to define the build rules for the application, including the dependencies and the steps required to create the Docker image. The main disadvantage of Bazel is that it doesn't automatically identify the dependencies of the application, requiring developers to manually specify them in the *BUILD* file.

Dockershrink

Dockershrink is an AI-powered tool that optimizes Docker images by analyzing the application and generating a Dockerfile or optimizing an existing one [?]. As far as I could determine, it only looks at Dockerfile misconfigurations and does not actually analyze the application to find its dependencies and it currently only supports NodeJS applications.

Slim

Slim, previously known as DockerSlim, is an open-source tool that automatically minimizes Docker images by analyzing the application and its dependencies, as well fixing common security issues [?]. It works by running the application in a container and monitoring its behavior using *ptrace*, *fanotify*, etc to identify the dependencies it uses at runtime, then it generates a new Docker image that only includes those dependencies. Additionally, it can also execute commands inside the testing container to ensure that all code paths are exercised, further refining the dependency list. As Slim takes an already existing image as input, it can be used with any application, regardless of the programming language or framework used. However, Slim has some limitations, such as difficulty in handling applications that require complex setup or configuration, and it may not always produce the smallest possible image due to its reliance on dynamic analysis, as well as potential issues with incomplete code path coverage. This however remains, as far as I could tell, the best tool available for minimizing container images at the moment.

3 My Work

Considering the disadvantages of existing tools for minimizing container images, I developed a new tool called *dockermimizer* that aims to address these issues.

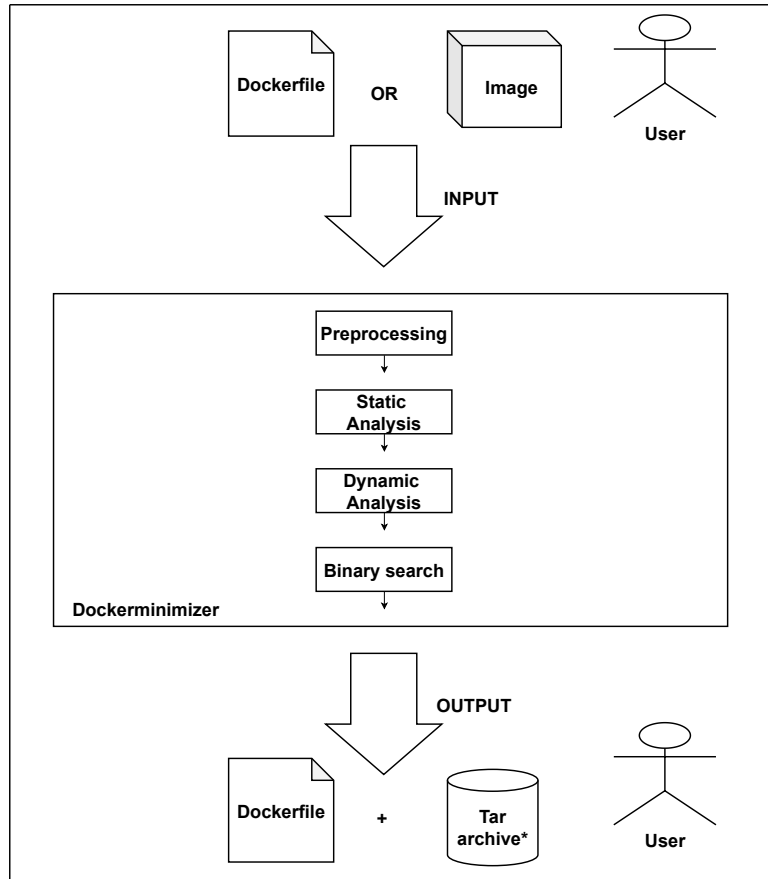


Figure 1: High-level architecture of the application

My tool is implemented in *Go* and is designed to be modular and extensible. It takes as input either a Dockerfile or an existing Docker image and performs static, dynamic analysis and binary search to minimize the application containerized, outputting the Dockerfile and if needed the tarball of all dependency files. It takes the following command-line arguments:

- **-file**: The path to the Dockerfile to be minimized.
- **-image**: The name of an images from a Docker registry to be minimized
- **-max_limit**: How many times should the binary search procedure be run for
- **-debug**: Enable logging of actions
- **-timeout**: How long should the minimal Docker container run before being declared as successful
- **-strace_path**: The path to a statically-linked strace binary

- **–binary_search:** Decide whether to continue the minimization process with a binary search if dynamic analysis failed

The tool expands from Slim by firstly performing static analysis to conserve processing resources and has a brute-force binary search approach in order to ensure that the minimization process terminates successfully even in cases where dynamic analysis fails.

Binary Search

Since this is a novel approach, I will briefly describe how the binary search procedure works.

The binary search procedure is a fail-safe mechanism that is used when the dynamic analysis fails to yield any results. It works by splitting the original image's filesystem into two halves and checking if the application can run successfully with only one of the halves. If it can, then the other half is split into two again and added to the first half. Once the application successfully runs, the files that remained in the second pile are discarded and the new files become the new baseline. This process is repeated until the maximum number of iterations is reached or the image cannot be minimized any further.

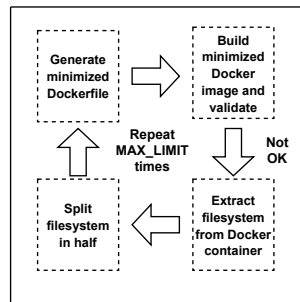


Figure 2: Binary search procedure for minimizing the application

4 Conclusion and Further Work

My tool is still a work in progress and there are several features that are missing or need improvement. I have created a comparison table between my tool and Slim in order to highlight the differences and similarities between the two tools.

Table 1: Comparison between Dockerminimizer and Slim

Feature	Dockerminimizer	Slim
Implementation Language	Go	Go
Analysis Technique	Static Analysis ('ldd') Dynamic Analysis ('strace') Binary Search (Brute Force)	Dynamic Analysis (Sensors/Probes) Static Metadata Analysis
Output Format	Minimized Dockerfile + Filesystem Tarball	Minified Docker Image ('.slim') (Opaque Binary)
Fallback Mechanism	Binary Search (Automated recovery of missing files)	Manual Configuration (User must manually include paths)
Base Image	FROM scratch	Custom Minified Distro
Transparency	High (User can inspect/edit the resulting Dockerfile)	Low (Black-box image generation)
Additional Features	None	Full Suite (xray, lint, debug, merge, registry, vulnerability)

The main advantage of my tool is the transparency of the resulting Dockerfile, which allows users to inspect and modify the minimized image as needed and the automated binary search procedure that ensures that the minimization process terminates successfully even in cases where dynamic analysis fails. However, in order to ensure the viability of the tool, I need to either implement some of the additional features that Slim offers or improve on the minimization process in order to achieve better results than Slim.

I have also identified several areas for further work, including:

- **Dynamic Analysis Improvements:** In addition to *strace* (which uses *ptrace*), I could explore other dynamic analysis techniques such as eBPF or systemtap to capture more comprehensive runtime information and potentially improve the accuracy of the minimization process.
- **Binary Search Optimization:** The current brute-force binary search approach can be time-consuming, especially for larger images. I could investigate more efficient search algorithms or heuristics to reduce the number of iterations needed to find the minimal set of files.
- **Security Analysis Integration:** Integrating security analysis tools to identify and remove vulnerable dependencies could enhance the security posture of the minimized images.
- **User Interface Enhancements:** Developing a more user-friendly interface, such as a web dashboard or CLI improvements, could make the tool more accessible to a wider range of users.

- **Support for Additional Container Formats:** Extending support to other container formats (e.g., OCI images) could broaden the applicability of the tool in different container ecosystems.
- **Application Path Analysis:** Implementing a more sophisticated application path analysis to better understand the dependencies and interactions between files could lead to more effective minimization.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. <https://arxiv.org/abs/1706.03762>, 2017.